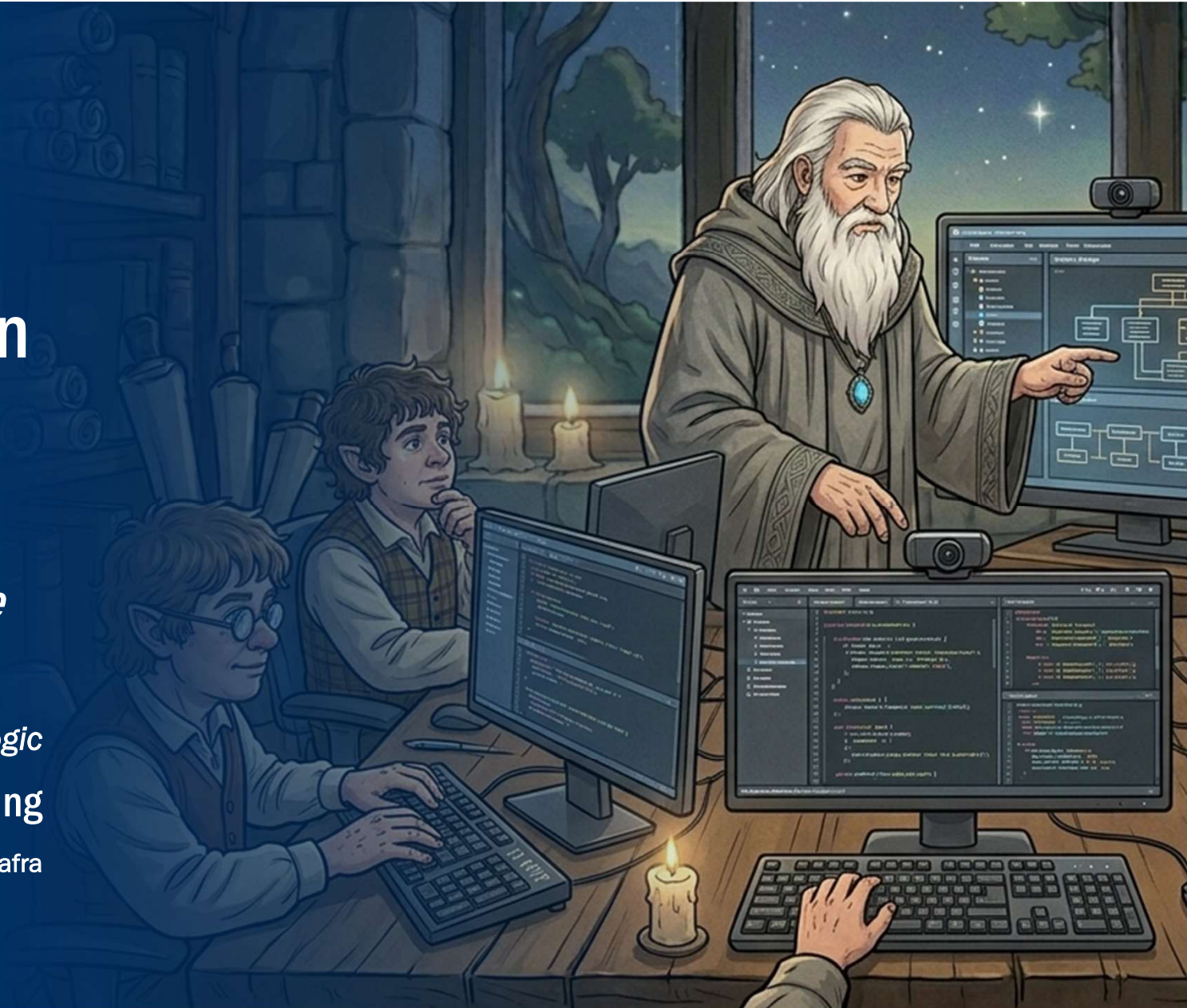


Application Design and Development (ADaD)

-The Fellowship of the Code

Phase 3: Application Logic
Modeling

Spring 2026, by Stefan Nafra



0. Agenda

1. Feedback
2. Recapitulation
3. Theory
4. Practice
5. Recapitulation
6. Feedback

Feedback

1. Phase Review



1. Phase Review

*„Despair is only for those who
see the end beyond all doubt.
We do not.“*

- Gandalf



Recapitulation

2. Phase Recapitulation



2. Phase Recapitulation

1. Roadmap
2. Flashback
3. Feedback Management
4. Practice: TFC: Representation
5. Knowledge Recapitulation: Quiz
6. Community Challenge (Meta Game)

2.1. Roadmap



2.2. Flashback

CONTENTS

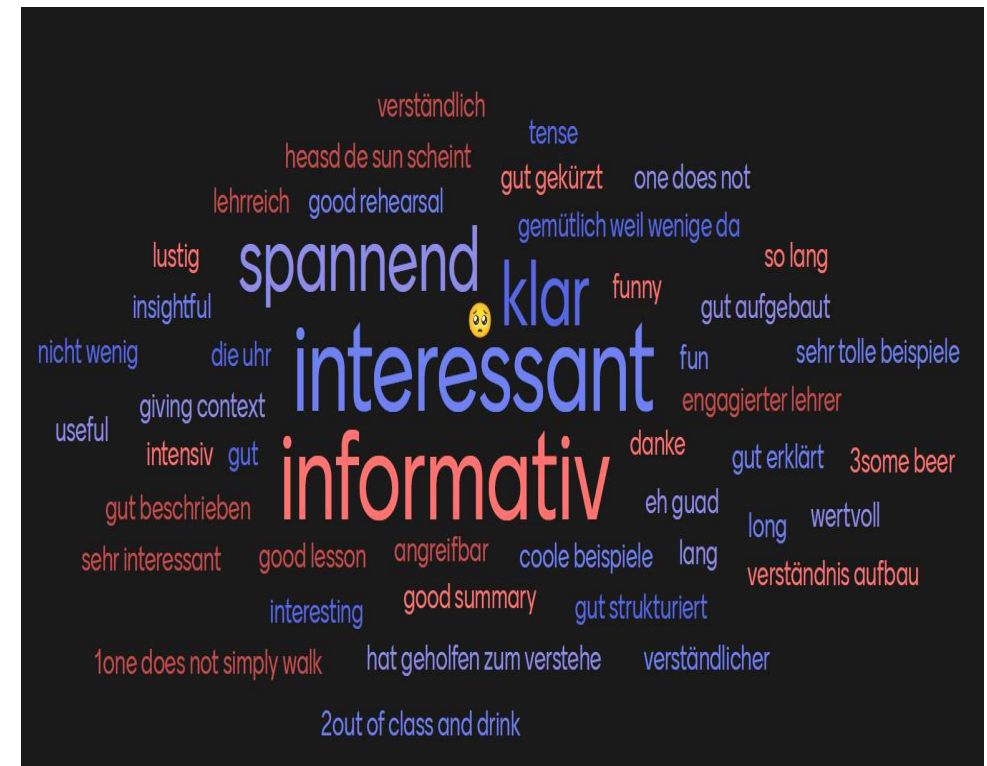
- **Web Development**
 - Web basics
 - HTML: HyperText Markup Language
 - CSS: Cascading Style Sheets
 - Tooling

TAKEAWAYS

- **Functionality of various web concepts**
 - Client-Server model, HTTP, URLs, APIs, ...
- **How to create a basic HTML document**
 - Elements & attributes, document anatomy, common elements, ...
- **How to use CSS and style HTML documents**
 - Ruleset anatomy, selectors, box model, responsive design, ...

2.3. Feedback Management

- Positive critics: Thanks <3
- Mixed critics:
 - “Quiz etwas chaotisch”
 - “Quiz am Anfang / Stoff-Wiederholung vor dem Quiz”
 - “Feedback verbraucht viel Zeit”
 - “Absprache mit anderen LektorInnen bezüglich überschneidenden Aufwänden”
 - “Online-Unterricht für bessere Stoffverteilung”
 - “Bitte kein Harry Potter im 4ten Semester / Slytherin sucks”
 - “Gruppenaufgaben sind schwierig zu organisieren / Mehr Einzelaufgaben”
 - “HdR-Recap am Anfang”
 - “GitHub-Einführung zu Beginn”
 - “Teilweise zu viel Material”
 - “Gemeinsam fortgehen” / “Gemeinsam D&D spielen” / “Gemeinsam HdR schauen” / “Gemeinsam Bierpong spielen”



"THE LORD OF THE RINGS"
TRILOGY IN 3 PANELS



THE OTHER END COMICS



NEIL KOHNEY

Source: <https://www.kohney.com/>



Source: 9Gag

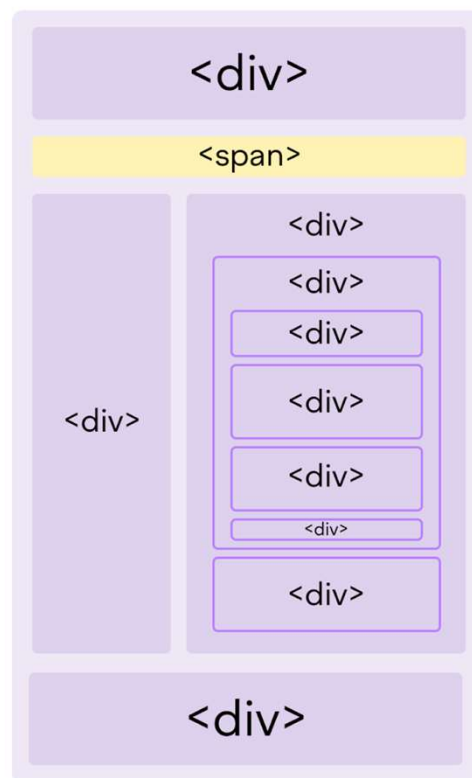
2.4. Practice: TFC: Representation

■ Lessons learned:

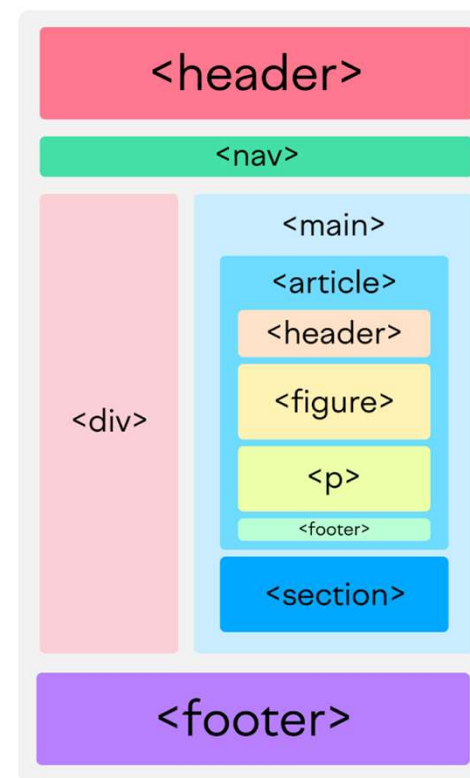
- *Read instructions: Do not waste valuable points by missing required details*
- **HTML & CSS:** Easy to learn but difficult to master
- **Reusability:** Think about the structure of your code (*DRY principle*)
- **Responsive design:** Be aware that multiple clients may be using the application
- **Obsolete elements:** Avoid outdated elements and techniques (e.g., *HTML tags for formatting instead of CSS - like using old tags as `<b>/<hr>/<center>` or `<table>` for layouting*)
- **CSS code:** Use external style sheets instead of inline CSS
- **Syntax:** HTML and CSS are forgiving (*does not fail-fast*) but errors are still errors and can lead to wrong behavior -> use a syntax checker (<https://validator.w3.org/nu/>)
- **Semantics:** Enable machines to interpret your code (e.g., *for Accessibility or SEO*)

What Is Semantic HTML?

Non-Semantic HTML



Semantic HTML



Source: <https://www.semrush.com/blog/seo-for-developers/>

2.5. Knowledge Recapitulation: Quiz

„You shall not pass.“
- Gandalf



Recommended watching:

<https://www.youtube.com/watch?v=3xYXUeSmb-Y>

2.5.1. Rules, Grading and Rewards

■ Rules:

- Only one answer can be correct
- Some questions are tricky (*some silly*)

■ Grading:

- Correct answers: 500 – 1.000 points
- Incorrect answers: 0 points
- Total: 10.000 points available

■ Rewards:

- > 6.000 points: 1.5 points (*per team*)
- > 4.000 points: 1 point (*per team*)
 - 5 points maximum (*per team*)

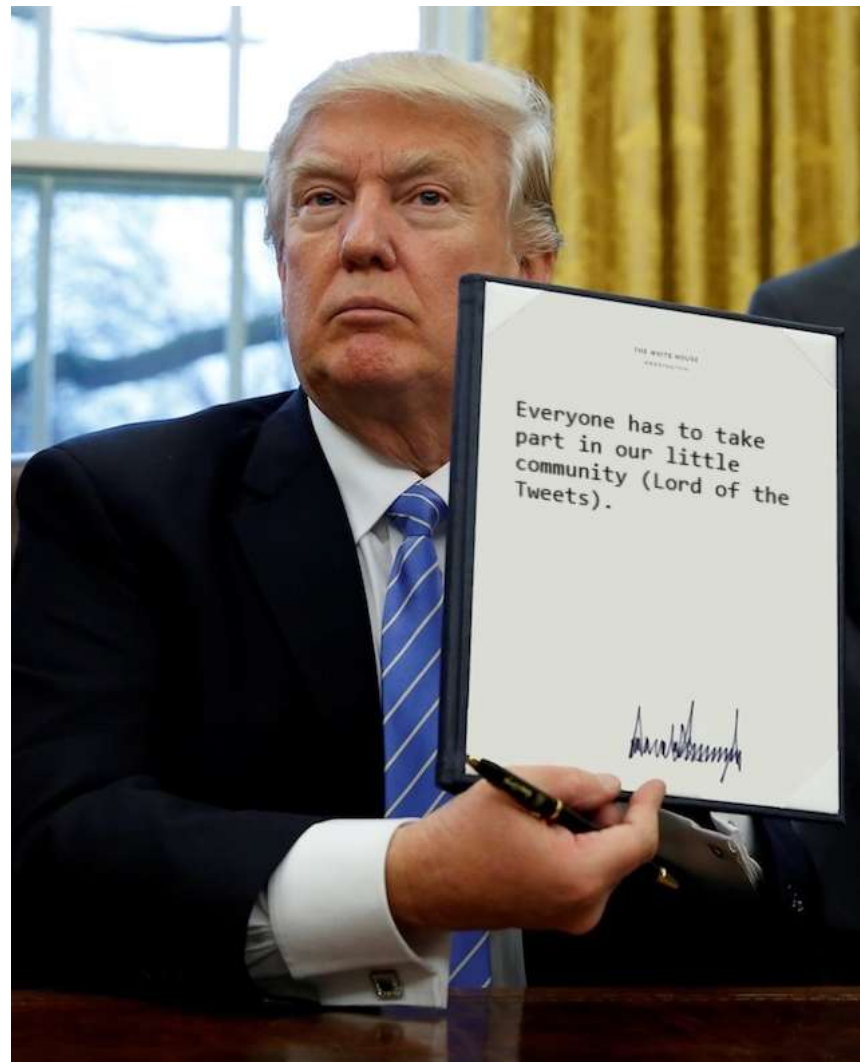
■ **Important:** Use your full name as your nickname (*I cannot assign you any points if you do not*)

2.6. Community Challenge (Meta Game)

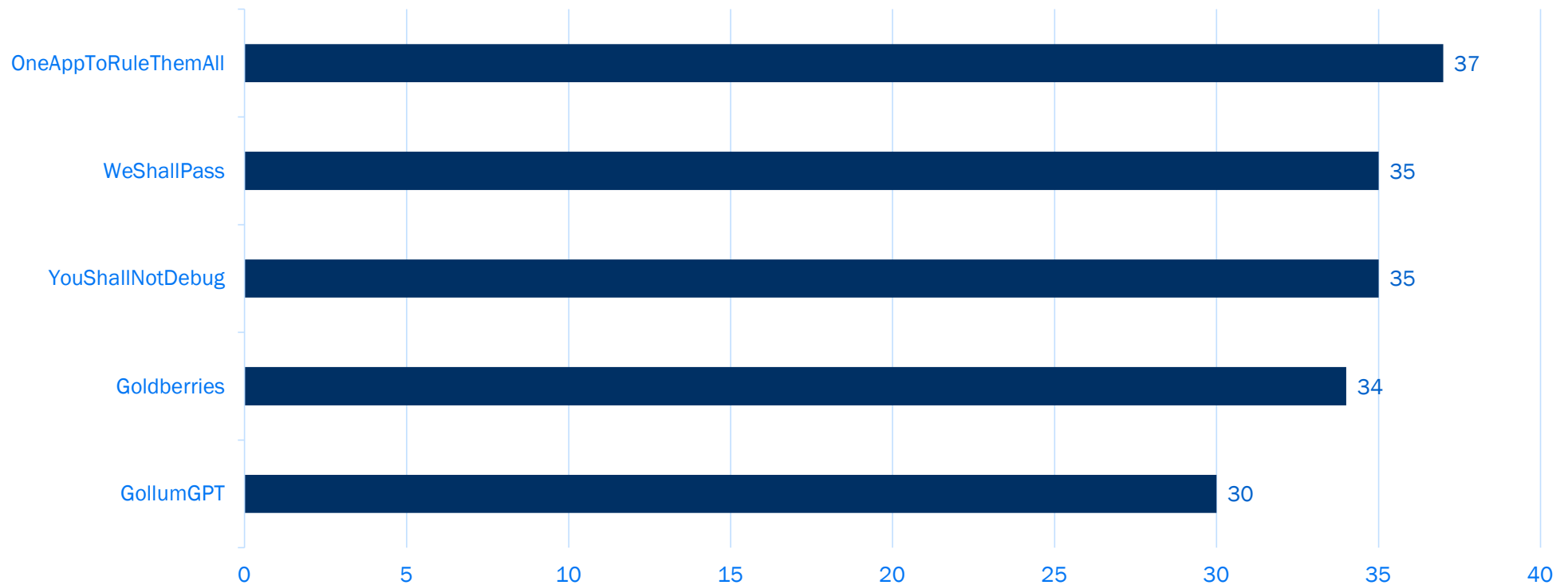
- Your activity on [Lord of the Tweets](#)
 - Tweets: X (X*)
 - Reactions: XX (XX*)
 - Comments: XX (XX*)

* = Last year





2.6.1. Activity Score





Break Time



- *“Gentleman, we do not stop before nightfall”*
- *“What about breakfast?”*
 - *“We already had it.”*
- *“We had one, yes. What about second breakfast?”*

Recommended watching:

<https://www.youtube.com/watch?v=gA8LV37QwxA>



Theory: Web Concepts

- 3. Web Basics
- 4. HTML: HyperText Markup Language
- 5. CSS: Cascading Style Sheets
- 6. Tooling

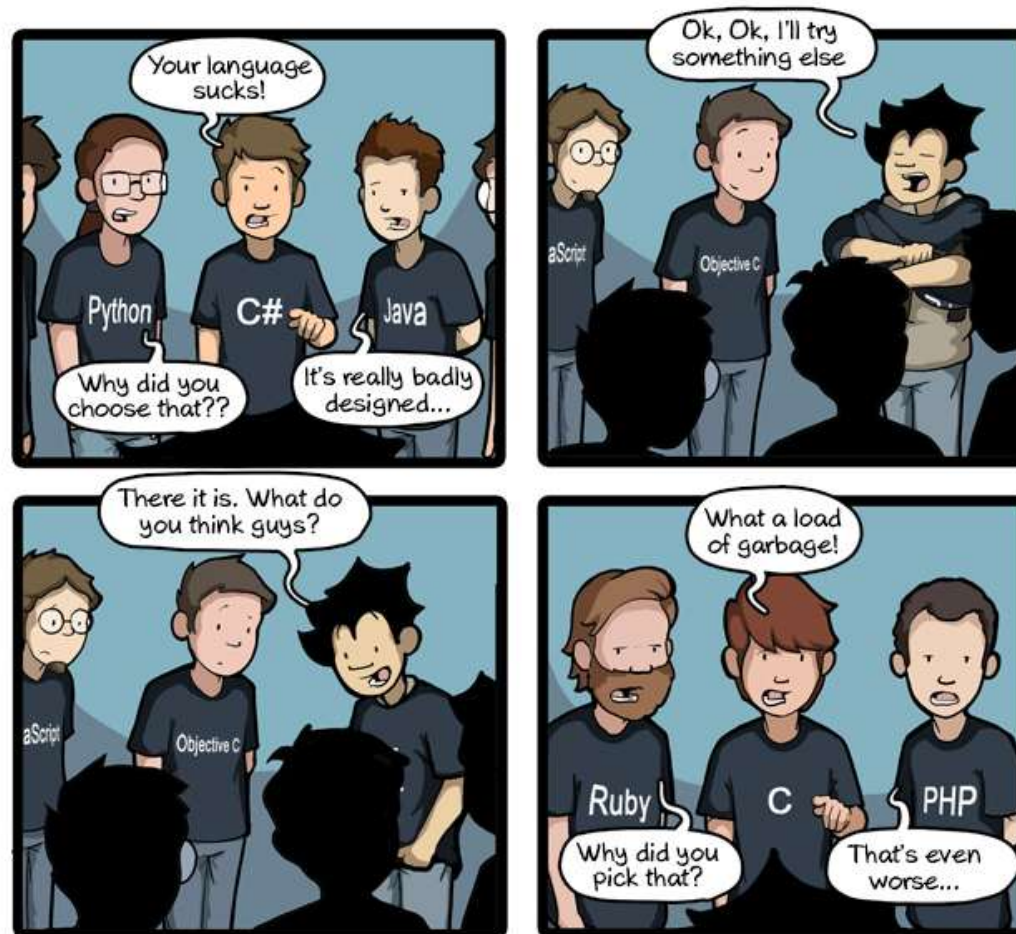
3.

Development Fundamentals & Concepts

1. Introduction
2. Variables
3. Operators
4. Conditional Statements
5. Loops
6. Functions

3.1. Introduction

- To build an application we can utilize multiple techniques, programming languages and tools
- Usually, a common body of general concepts must be understood and utilized for creating software applications (*independent of the specific programming language*)



Source: <https://www.commitstrip.com/>

3.2. Variables

- While programming, we need to store data for further usage
- Most common: Variables
 - Variables are containers for values
 - Example: *fellowship* = 9
 - fellowship: Name of the variable
 - 9: Value of the variable
- We can access and update the contents later (*depending on the scope*)

Recommended reading: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/Variables/

```
1  /* -----  
2  | 1. Variables  
3  | ----- */  
4  
5  let a;  
6  console.log('The value of a is ' + a); // The value of a is undefined  
7  
8  const b = 'Samwise';  
9  console.log('The value of c is ' + b); // The value of b is Samwise  
10  
11 let b = 'Frodo';  
12 console.log('The value of c is ' + b); // SyntaxError: Identifier 'b' has already been declared  
13  
14 console.log('The value of c is ' + c); // ReferenceError: c is not defined
```

Recommended reading: <https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/introduction.html>

3.2.1. Variable Types

- **Number:** An integer or floating point number (e.g., 9 or 3.14159)
- **String:** A sequence of characters that represent a text value (e.g., “Hobbit”)
- **Boolean:** True and false
- **Indexed arrays:** Multiple values in list form (e.g., [“Hobbit”, “Human”, “Orc”])
- **Associative arrays:** Multiple pairs of name and value in list form (e.g., { “Race”: { “Hobbit”: “Frodo”, “Human”: “Boromir” }, “Enemies”: “Sauron” }

Recommended reading: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/Variables/



Source: <https://medium.com/@daboigbae/the-best-programming-memes-this-week-episode-002-85fc27737f78>


```
16  /* -----
17  |    2. Variable Types
18  | ----- */
19
20  /* Number */
21  let myNumber = 9;
22
23  /* String */
24  let myString = 'Boil em, mash em, stick em in a stew.';
25
26  /* Boolean */
27  let myBoolean = true;
28
29  /* Indexed array */
30  let myArray = ['Lord', 'of', 'the', 'Rings', 'is', 'awesome'];
31
32  /* Associative array */
33  let myObject = {
34    Movie1: {
35      Title: 'Lord of the Rings',
36      Rating: 'Awesome'
37    },
38    Movie2: {
39      Title: 'Leon',
40      Rating: 'Amazing'
41    },
42    Movie3: {
43      Title: 'Nightmare before Christmas',
44      Rating: 'I love it'
45    }
46  }
```

Recommended reading: <https://estebanthewhite.github.io>

3.3. Operators

■ Arithmetic operators:

- Calculating sums
- +, -, *, /, %, and **

■ Assignment operators:

- Assign a value to a variable
- =
- +=, -=, *=, /=

■ Comparison operators:

- Testing conditions (true / false)
- ===, !==, <, >, <=, >=

Recommended reading: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/Math/

```
49  /* -----  
50      3. Operators  
51  ----- */  
52  
53  /* Arithmetic operators */  
54  console.log(5 * 5) // 25;  
55  
56  /* Assignment operators */  
57  let y = 15;  
58  console.log(y) // 5;  
59  y = 10;  
60  console.log(y) // 10;  
61  y += 10;  
62  console.log(y) // 20;  
63  
64  /* Comparison operators */  
65  console.log(y === 20) // true;  
66  console.log(y !== 20) // false;  
67  console.log(y >= 20) // true;
```

Recommended reading: <https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/introduction.html>

3.4. Conditional Statements

- Control the flow within your application by checking for certain conditions and decide how to react
- Supported by using the if and else construct
 - Can be nested and chained

Recommended reading: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling/


```
69  /* -----  
70  | 4. Conditional Statements  
71  | ----- */  
72  
73  let z = 20;  
74  
75  if (z === 10) {  
76      console.log('z equals 10 - it stores the value of ' + z);  
77  } else {  
78      console.log('z does not equal 10 - it stores the value of ' + z);  
79      if (z > 50) {  
80          console.log('z is larger 50');  
81      } else {  
82          console.log('z is smaller or equal 50');  
83      }  
84  }
```

Recommended reading: <https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/introduction.html>

3.5. Loops

- Loops offer a quick and easy way to do something repeatedly
 - Example: Access each entry within a list
- Multiple types of loops exist:
 - While
 - Do while
 - For
 - For each
 - ...

Recommended reading: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration/

```
88  /* -----
89  |   5. Loops
90  | ----- */
91
92  /* While */
93  let x = 1;
94  while (x <= 1000) {
95  |   console.log(x); // nothing
96  |   x = x + 1;
97  | }
98
99  /* For */
100 for (let i = 1; i <= 100; i++) {
101 |   console.log(i); // 1, 2, 3, 4, 5
102 | }
103
104 /* For in */
105 let friends = {
106 |   A: 'Samwise',
107 |   B: 'Frodo',
108 |   C: 'Gimli'
109 | };
110 for (let names in friends) {
111 |   console.log(names); // A, B, C
112 |   console.log(friends[names]); // Samwise, Frodo, Gimli
113 | }
```

Recommended reading: <https://estebanthewhite.com/>

3.6. Functions

- Set of statements to perform a task
 - Example: Calculates a value based on some input parameters and returns it afterwards
- Usually consumes an input and returns an output
- A function definition consists of:
 - Function keyword
 - Name of the function (*can be anonymous*)
 - List of parameters (*enclosed in parentheses and separated by commas*)
 - Statements that define the function (*enclosed in curly brackets*)

Recommended reading: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions/>

```
115  /* -----  
116  |    6. Functions  
117  | ----- */  
118  
119  function myFunction(var1, var2) {  
120  |    console.log(var1 + ' is stronger than ' + var2);  
121  | }  
122  myFunction('Samwise', 'Aragorn'); // Samwise is stronger than Aragorn  
123
```

Recommended reading: <https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/introduction.html>

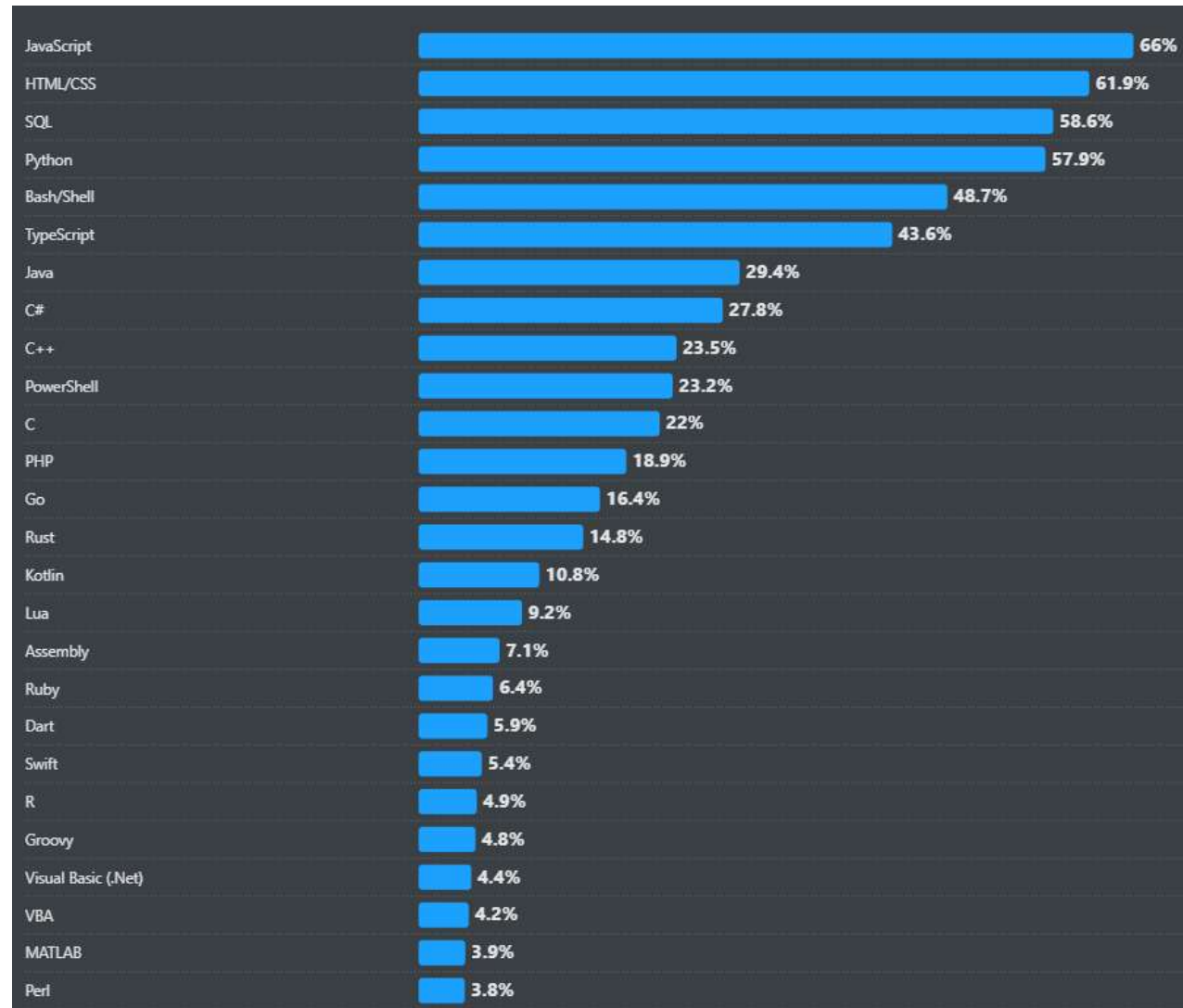
4. JavaScript

1. Introduction
2. Document Object Model (DOM)
3. Debugging
4. Self-Study Material
5. Inspiration

4.1. Introduction

- Essential technology for building web applications and enabling dynamic behavior
 - All modern browser support and run JavaScript code in dedicated engines
- High-level, usually just-in-time compiled, and multi-paradigm language and comes with:
 - Curly-bracket syntax
 - Dynamic typing
 - ...

Recommended reading: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/>



Source: <https://survey.stackoverflow.co/2025/technology#1-programming-scripting-and-markup-languages>

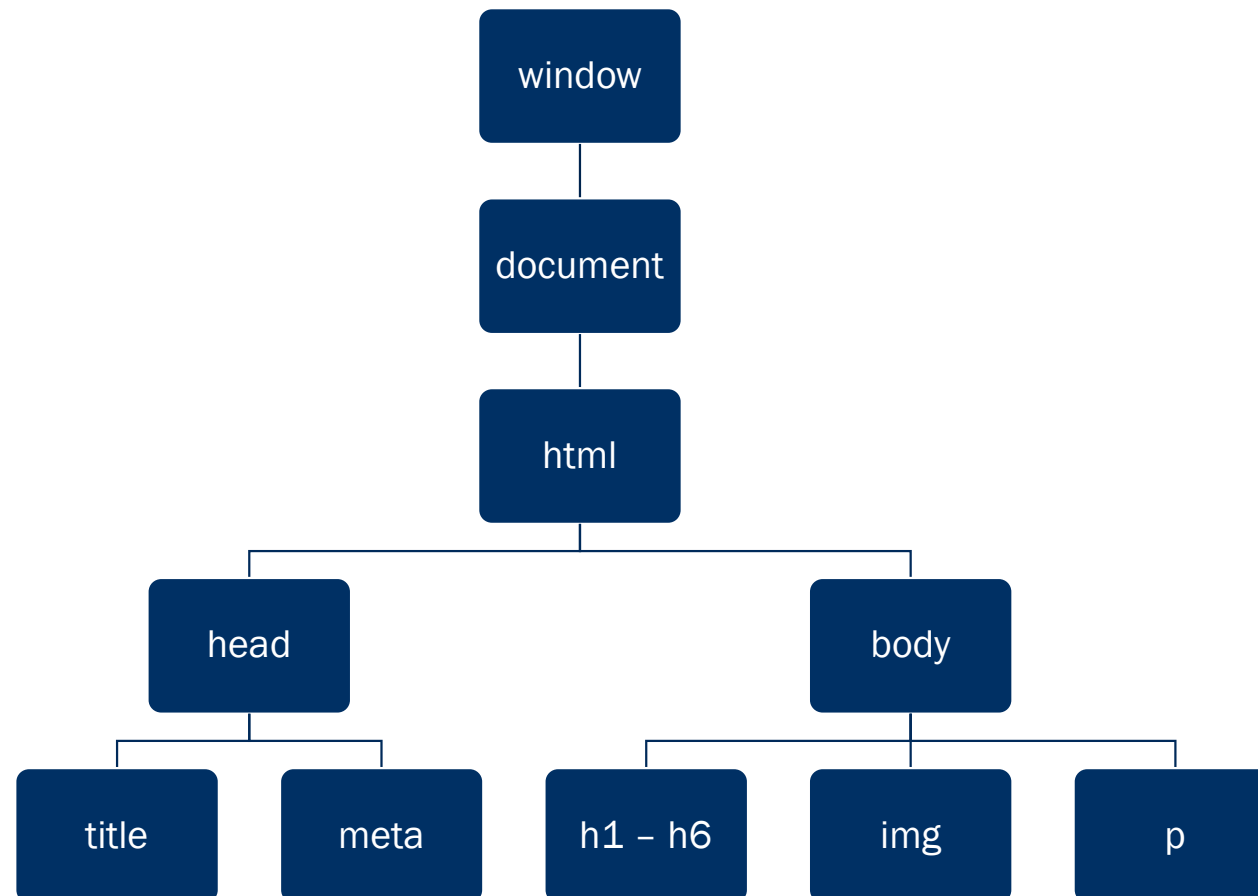
THE LORD OF THE RINGS ANALOGY TO PROGRAMMING LANGUAGES

Python The Ent	Java Gandalf	C One Ring	C++ Saruman	JavaScript Hobbit	C# Elf	Ruby Man (Middle Earth)	PHP Orc	Objective-C Smog
								
DIFFICULTY ★★★★★	DIFFICULTY ★★★★★	DIFFICULTY ★★★★★	DIFFICULTY ★★★★★	DIFFICULTY ★★★★★	DIFFICULTY ★★★★★	DIFFICULTY ★★★★★	DIFFICULTY ★★★★★	DIFFICULTY ★★★★★
								
Help little Hobbits (beginners) to understand programming concepts Help Wizards (computer scientists) to conduct researches Widely regarded as the best programming language for beginners Easiest to learn Widely used in scientific, technical & academic field, i.e. Artificial Intelligence You can build website using Django, a popular Python web framework	Words peace & works with everyone (portable) Very popular on all platforms, OS, and devices due to its portability One of the most in demand & highest paying programming languages Slogan: write once, work everywhere	The power of C is known to them all Everyone wants to get its Power Unique franchise of programming language One of the oldest and most widely used language in the world Popular language for system and hardware programming A subset of C++ except the little details	Everyone thinks that he is the good guy But once you get to know him, you will realize he wants the power, not good deeds Complex version of C with a lot more features Widely used for developing games, industrial and performance-critical applications Learning C++ is like learning how to manufacture, assemble, and drive a car Recommended only if you have a mentor to guide you	Frequently underestimated (powerful) Well known for the slow, gentle life of the Shire (web browsers) "Java and JavaScript are similar like Car and Carpet are similar" - Greg Mewall Most popular client-side web scripting language A must learn for front-end web developer (HTML and CSS as well) One of the hottest programming language now, due to its increasing popularity as server-side language (node.js)	Beautiful creature (language), but stays in their land, Rivendel (Microsoft Platform) A popular choice for enterprise to create websites and Windows application using .NET framework Can be used to build website with ASP.NET, a web framework from Microsoft Similar to Java in basic syntax and some features Learn C# instead of Java if you are targeting to work on Windows platform only	Very emotional creature They (home Ruby developers) feel they are superior & need to rule the Middle Earth Mostly known for its popular web framework, Ruby on Rails Focuses on getting things done Designed for fun and productive coding Best for fun and personal projects, startups, and rapid development	Ugly guy (language) and doesn't respect the rules (Inconsistent and unpredictable) Big headache to those (developers) to manage them (coding) Yet still dominates the Middle-earth (most popular web scripting language) Suitable for building small and simple sites within a short time frame Supported by almost every web hosting services with lower price	Lovely and looks good Primary language used by Apple for Mac OS X & iOS Choose this if you want to focus on developing iOS or OS X apps only Consider to learn Swift (newly introduced by Apple in 2014) as your next language
POPULARITY ★★★★★	POPULARITY ★★★★★	POPULARITY ★★★★★	POPULARITY ★★★★★	POPULARITY ★★★★★	POPULARITY ★★★★★	POPULARITY ★★★★★	POPULARITY ★★★★★	POPULARITY ★★★★★
USED TO BUILD YouTube, Instagram, Spotify	USED TO BUILD Gmail, Microsoft, Most Android Apps, Enterprise applications	USED TO BUILD Operating systems and hardware	USED TO BUILD Operating systems, hardware, and browsers	USED TO BUILD Paypal, front-end of majority websites	USED TO BUILD Enterprise and Windows applications	USED TO BUILD Hulu, Groupm, Stackshare	USED TO BUILD WordPress, Wikipedia, Flickr	USED TO BUILD Most iOS Apps and part of Mac OS X
AVG. SALARY \$107,000	AVG. SALARY \$102,000	AVG. SALARY \$102,000	AVG. SALARY \$104,000	AVG. SALARY \$99,000	AVG. SALARY \$94,000	AVG. SALARY \$107,000	AVG. SALARY \$89,000	AVG. SALARY \$107,000

Source: <https://imgur.com/I5qmY90>

4.2. Document Object Model (DOM)

- Structure and hierarchy of an HTML document
 - Form of a logical tree
 - The tree branches ending in nodes
- DOM methods enable the programmatic access to the tree (e.g., *through JavaScript*)
 - Allowing to change the structure, style or content of a document
 - Event handlers can be attached to Nodes



4.2.1. DOM API: Manipulating HTML and CSS

- The DOM API is used for manipulating HTML and CSS
 - Creating, removing and changing HTML and CSS (*dynamically*)
- HTML manipulation:
 - Select: `document.querySelector('#id');`
 - Create: `document.createElement('p');`
 - Add: `appendChild(element);`
- DOM events:
 - Change: `document.querySelector('.class').addEventListener('click', function () {
– // do something...`
 - `});`

Recommended reading: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Manipulating_documents/

```
124  /* -----  
125      7. DOM Manipulation  
126  ----- */  
127  
128  /* Create HTML elements */  
129  let link = document.createElement('a');  
130  link.textContent = 'Ardapedia';  
131  link.href = 'https://www.ardapedia.org/wiki/Hauptseite';  
132  
133  /* Select and add HTML elements */  
134  document.querySelector('.result').appendChild(link);  
135  
136  /* Change style attributes */  
137  link.style.color = 'red';
```

Recommended reading: <https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/introduction.html>

4.3. Debugging

- Most browsers come with tools to support with web development (*F12*)
- Support in-depth analysis of the interpreted code:
 - DOM
 - CSS styles
 - Events
 - Cookies
 - Request & response messages
- “Elements” tab for inspecting HTML & CSS and “Sources” tab for debugging JavaScript

Recommended reading: <https://javascript.info/debugging-chrome/>

Programmers while reviewing the codes



Source: <https://twitter.com/pr0grammerhum0r/status/1425472096117874693/>

4.4. Self-Study Material

- Tutorial: https://nafra.at/adad_st2026/tutorial_javascript.html
- Command reference: PDF-Document in Files > P3
- Best practices:
 - <https://javascript.info/coding-style/>
 - https://www.w3schools.com/js/js_conventions.asp
 - https://www.w3schools.com/js/js_best_practices.asp
- Troubleshooting:
 - https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/What_went_wrong/
 - <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/>
 - <https://javascript.info/debugging-chrome/>
 - https://www.w3schools.com/js/js_mistakes.asp

4.5. Inspiration

- JavaScript insights:
 - <https://2025.stateofjs.com/en-US>
- Technology insights:
 - <https://survey.stackoverflow.co/2025/>
- Webdesign inspiration:
 - <https://omr.com/de/reviews/contenthub/website-design-inspirationen>
- Cheat sheets:
 - <https://overapi.com/>
- CSS selectors:
 - <https://webdevsimplified.com/specificity-cheat-sheet.html>



Break Time



- *“Gentleman, we do not stop before nightfall”*
- *“What about breakfast?”*
 - *“We already had it.”*
- *“We had one, yes. What about second breakfast?”*

Recommended watching:

<https://www.youtube.com/watch?v=gA8LV37QwxA>

Practice

5. JavaScript



5. JavaScript

1. Introduction
2. Random Number Game
3. Complaint Form

5.1. Introduction

- I will demonstrate some of the fundamentals and concepts of JavaScript + Repl.it

Recommended reading: <https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/introduction.html>



Break Time

- “Gentleman, we do not stop before nightfall”
- *“What about breakfast?”*
 - “We already had it.”
- *“We had one, yes. What about second breakfast?”*

Recommended watching:

<https://www.youtube.com/watch?v=gA8LV37QwxA>

5.2. Random Number Game

- A button executing the following function:
 - Calculate a random number between X – Y
 - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random/
 - If the random number equals the number from the input field print a success message on that page

Recommended reading: <https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/randomnumbergame.html>

5.3. Complaint Form

- A button executing the following function:
 - Print all data provided by the user

Recommended reading: <https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/contactform.html>

Recapitulation

- 6. Lecture Recapitulation
- 7. Self-Study Assignments



6.

Lecture

Recapitulation

- 1. Flashback
- 2. Quiz

6.1. Flashback

CONTENTS

- Development Advanced
 - Development fundamentals & concepts
 - JavaScript

TAKEAWAYS

- Functionality of various programming concepts
 - Variables, Operators, Conditional statements, Loops and Functions
- Introduction and first steps in programming on the example of JavaScript

6.2. Quiz

„You shall not pass.“
- Gandalf



Recommended watching:

<https://www.youtube.com/watch?v=3xYXUeSmb-Y>

6.2.1. JavaScript: Explain the purpose of variables

- Variables are containers for values
- Example: `fellowship = 9`
 - `fellowship`: Name of the variable
 - `9`: Value of the variable
- We can access and update the contents later (*depending on the scope*)

Recommended reading: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/Variables/

6.2.2. JavaScript: Name some common variable types

- **Number:** An integer or floating point number (e.g., 9 or 3.14159)
- **String:** A sequence of characters that represent a text value (e.g., “Hobbit”)
- **Boolean:** True and false
- **Indexed arrays:** Multiple values in list form (e.g., [“Hobbit”, “Human”, “Orc”])
- **Associative arrays:** Multiple pairs of name and value in list form (e.g., { “Race”: { “Hobbit”: “Frodo”, “Human”: “Boromir” }, “Enemies”: “Sauron” }

Recommended reading: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/Variables/

6.2.3. JavaScript: Explain the purpose of conditional statements

- Control the flow within your application by checking for certain conditions and decide how to react
- Supported by using the if and else construct
 - Can be nested and chained

Recommended reading: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling/

6.2.4. JavaScript: Explain the purpose of loops

- Loops offer a quick and easy way to do something repeatedly
 - Example: Access each entry within a list
- Multiple types of loops exist:
 - While
 - Do while
 - For
 - For each
 - ...

Recommended reading: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration/

6.2.5. JavaScript: Explain the purpose of the DOM API

- The DOM API is used for manipulating HTML and CSS
 - Creating, removing and changing HTML and CSS (*dynamically*)
- HTML manipulation:
 - Select: `document.querySelector('#id');`
 - Create: `document.createElement('p');`
 - Add: `appendChild(element);`
- DOM events:
 - Change: `document.querySelector('.class').addEventListener('click', function () {
– // do something...`
 - `});`

Recommended reading: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Manipulating_documents/

7. Self-Study Assignments

#	TITLE	TYPE	DUE DATE	TASKS	TOOLING
11	Reading the Runes	PRAC	26.05.2026	• Explain existing logic	• GitHub & GitHub Codespaces
12	TFC: Logic & State	PRAC	26.05.2026	• Implement a dynamic interface	• GitHub & GitHub Codespaces
13	TRDoW: Chapter IV – When Things React	PRAC	29.05.2026	• Document & reflect • Share & care • Prepare	• GitHub & GitHub Codespaces

Feedback

8. Lecture Review





8. Lecture Review

*„Despair is only for those who
see the end beyond all doubt.
We do not.“*

- Gandalf